

RemAining Relevant

How Economists Can Evolve Before AI Eats Their Lunch

Benjamin S. Skrainka
Principal Economist
skrainka@amazon.com
[Amazon, Inc.](#)

February 12, 2026

Economics and Generative AI

This is a big topic and could mean:

- How AI affects society and the economy
- Using AI to increase productivity, scope, and scale
- Using deep learning techniques in Economic models

I will focus on using AI tools to super-charge the workflow for Economics

The AI-Powered Economist

Economists risk obsolescence if we don't update our workflow to use Gen AI:

- Convergence of job families into a “Super IC”
- Increased impact and expanded frontier of possibilities
- Reduced dependencies on other specialists (e.g., engineers, IT, etc.)
- Must embrace short term pain to integrate AI into our workflow

Eight Levels of AI Adoption by Steve Yegge (1/2)

- Level 1: no AI
- Level 2: Coding agent in your IDE, permissions turned on
- Level 3: Coding agent in IDE, “YOLO mode.” Your trust is going up.
- Level 4: you’re starting to not look at the diffs anymore, but at what the agent is doing. You’re not reviewing as much, you’re letting more of it through, and you’re really focused on the conversation with the agent.

Eight Levels (2/2)

- Level 5: your approach is: “I just want the agent and I’ll look at the code in my IDE later, but I’m not coding with my IDE”.
- Level 6: several agents. You’re bored because your agent’s busy and you want to do something, so you fire up another agent, then another. And you find yourself just multiplexing between them, and you can’t “leave” [you start to get addicted to using more agents.]
- Level 7: 10+ agents, managed by hand. This is where you typically say “oh gosh, I’ve made a mess! I accidentally texted the wrong agent and didn’t realize. How do I coordinate all these agents? What if Claude Code could run Claude Code?”
- Level 8: you build your own orchestrator to coordinate more agents”.

Agenda

Today, I will demo a couple ways AI is changing how Economists work:

- Everyday tasks using [Kiro CLI](#)
 - ▶ Working with documents (summary, assessment, writing)
 - ▶ Eliminating toil (convert whiteboard images to markdown)
 - ▶ Assessing submissions for a poster session
- Performing research and automation using [Quick Suite](#)
- Building code to answer research questions using *prompt driven development* using [Kiro CLI](#) or [Kiro](#)

Helpful things to install for this demo

- Download and install [Kiro CLI](#).
- Verify Kiro will run a simple prompt
- Ensure you can access Git Hub: I will distribute materials this way
- Ensure you have either a working Python (such as `conda/mamba`) or R installation

Optional helpful things to install

Optional: On OS/X,

- Install [iTerm2](#), which will make using the Terminal/CLI much more pleasant.
- Install [Home Brew](#) (aka 'brew') which will provide Unix-ish commands

Optional: On Windows,

- Install Windows terminal + widget or equivalent (such as Chocolatey)
- Note: I do not use Windows so YMMV

Key Terms

- **Large Language Model (LLM)**: a stateless, non-deterministic predictive algorithm—essentially a model architecture plus weights
- **Model Context Protocol (MCP)**: an open standard that enables LLMs to connect with external tools, data sources, and services
- **Tool**: a service an LLM can call using Model Context Protocol (MCP) to complete a task such as reading a PDF
- **Skill**: another name for a tool
- **Prompt**: instructions you provide to perform some task
- **Context**: additional input to nudge the LLM to produce relevant results (a prior and constraints)

Prompt Driven Development

Definition: An iterative approach where developers describe desired outcomes in natural language prompts, and AI generates code incrementally through conversation.

Key Characteristics:

- Exploratory and adaptive—requirements emerge through dialogue
- Low upfront investment; fast prototyping
- Code evolves via refinement cycles

Why It Suits Research:

- Supports hypothesis-driven exploration
- Accommodates changing requirements as understanding deepens
- Reduces boilerplate, letting researchers focus on ideas

Prompt vs. Spec Driven Development

Prompt Driven

Requirements discovered iteratively
Flexible, conversational
Best for exploration

Spec Driven

Requirements defined upfront
Formal, structured specifications
Best for well-understood problems

Agent Reasoning Loop

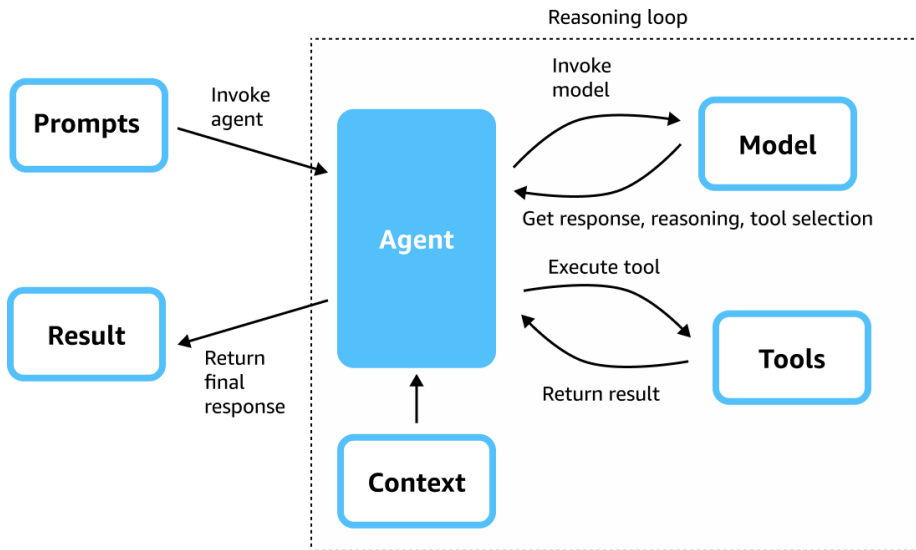


Figure 1: Agent Reasoning Loop

Warning: Ownership and Quality

You remain the owner of all work you create with Gen AI:

- Trust but verify
- Don't cowboy push some vibe coded AI slop to prod. . . .
- You are still responsible for quality and ethical standards of your work
- Don't outsource your thinking to AI, use it to accelerate your workflow
- Fact-check every fact at the original source